Database
(SOC-3020)

Alessandro Agostini[1,2]

aagostini@inha.ac.kr
agostini@disi.unitn.it

[1]LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

[2]DISI, University of Trento, Italy

WEEK 5





DATABASE

SQL: Introduction

WEEK 5

Code: W5.L1L2[QUIZ#1]

Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)



Any question from you?

...

MySQL

Installation Guidelines (*reminder*)

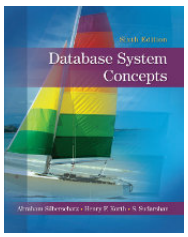
cf. Week 2

MySQL should be used by students to execute all queries proposed.



Relational Operations

Ch 2, T; section 2.6





Relational Operations

- We concentrate on the following operations:
  - selection of tuples/rows,
  - projection (or: selection of attributes/columns),
  - natural join,
  - Cartesian product,
  - union,
  - set difference,
  - intersection.
- In this lecture, we'll express each operation by using the language of Relational Algebra.
  - Successively, we'll use SQL.





1. Select – selection of rows

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

A	B	C	D
α	α	1	7
β	β	23	10





1. Select – selection of rows

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \text{ and } D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10





2. Project – selection of columns

- Relation *r*

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2





2. Project – selection of columns

- Relation *r*

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- $\prod_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2





3. Natural Join

- Relations *r, s*:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

- *r* + *s*:

ID	name	salary	dept_name	building	budget
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
12121	Wu	90000	Finance	Painter	120000
15151	Mozart	40000	Music	Packard	80000
22222	Einstein	95000	Physics	Watson	70000
32343	El Said	60000	History	Painter	50000
33456	Gold	87000	Physics	Watson	70000
45565	Katz	75000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
76543	Singh	80000	Finance	Painter	120000
76766	Crick	72000	Biology	Watson	90000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000





4. Join – Cartesian Product

- Relations *r, s*:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

- *r* x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b





5. Union of two relations

- Relations *r, s:*

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- r ∪ s:

A	B
α	1
α	2
β	1
β	3

Note: the two relations must be "similar", that is, of the same logical schema





6. Set difference of two relations

- Relations *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- *r* – *s*:

A	B
α	1
β	1

Note: the two relations must be "similar", that is, of the same logical schema





7. Intersection of two relations

- Relation *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- *r* ∩ *s*

Note: the two relations must be "similar", that is, of the same logical schema

A	B
α	2

Note: $r \cap s = r - (r - s)$



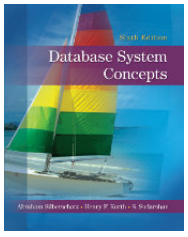
Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)

Relational Query Languages

or: Languages to "query" (i.e., manage) Relations

Ch 2, T; section 2.5



Query Languages

- Def. A query language is a language by which a user can access/update/delete data in a DBMS.



Languages' Paradigms

- Two typical kinds ("programming paradigms") of query languages are:
  - **Procedural**: the user instructs the system to perform a sequence of operations on the database to compute the result.
  - **Non-procedural**: the user describes the desired result without defining the "how".





Relational Query Languages

- "Pure" relational query languages:
  - Relational algebra (*procedural*)
  - Tuple relational calculus (*non-procedural*)
  - Domain relational calculus (*non-procedural*)
- Above languages are equivalent in computing power and expressiveness (equivalent to FOL - First Order Logic).
  - SQL (*commercial, non-procedural*)
- SQL is more expressive; based on pure languages



Remark 1

- There are many relational DB query languages in use, either commercially or experimentally.
  - We study the most widely used : SQL.
- Although SQL is known as a "*query* language", it contains both DDL + DML components.
- This course does NOT provide a users' guide for SQL, but only fundamental constructs and concepts. SQL's implementations *do* differ…



Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)

SQL

Structured **Q**uery **L**anguage



History

- IBM SEQUEL (Structured English Query Language), a language developed as part of System R project at the IBM San Jose Research Laboratory (Chamberlin & Boyce, 1974)
- Renamed Structured Query Language (SQL)
- ANSI and ISO standard SQL:
  - SQL-86
  - SQL-89
  - SQL-92
  - SQL:1999 (language name became Y2K compliant!)
  - SQL:2003 on
- Commercial systems offer most, if not all, SQL-92 features, plus varying feature sets from later standards and special proprietary features.
  - *Not all examples here may work on your particular system!*



SEQUEL: A STRUCTURED ENGLISH QUERY LANGUAGE

by

Donald D. Chamberlin
Raymond F. Boyce

IBM Research Laboratory
San Jose, California

ABSTRACT: In this paper we present the data manipulation facility for a structured English query language (SEQUEL) which can be used for accessing data in an integrated relational data base. Without resorting to the concepts of bound variables and quantifiers SEQUEL identifies a set of simple operations on tabular structures, which can be shown to be of equivalent power to the first order predicate calculus. A SEQUEL user is presented with a consistent set of keyword English templates which reflect how people use tables to obtain information. Moreover, the SEQUEL user is able to compose these basic templates in a structured manner in order to form more complex queries. SEQUEL is intended as a data base sublanguage for both the professional programmer and the more infrequent data base user.



SQL-DDL

Data **D**efinition **L**anguage



Data-Definition Language (DDL)

The SQL data-definition language (DDL) allows the specification of information about relations, including:

- The schema for each relation.
- The domain and type of values associated with each attribute.
- Integrity constraints
- Other information such as:
  - The set of indices to be maintained for each relations.
  - Security and authorization information for each relation.
  - The physical storage structure of each relation on disk.
- Output of DDL stored into Data Dictionary (or: System Catalog) as specific DB data called metadata.

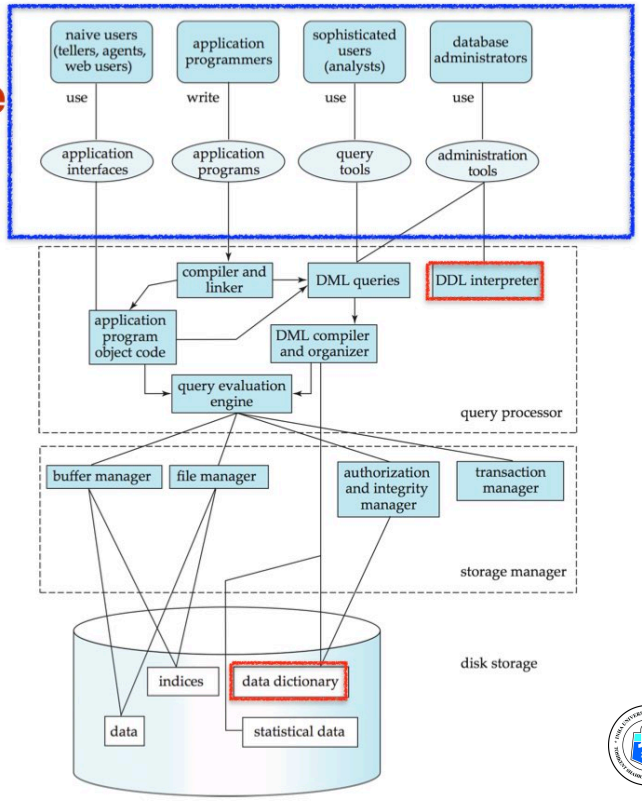




Reminder: DDL in Architecture

users and administrators

internal components







Reminder: Attributes' Domain and Type

- The set of allowed values for each attribute in a relation is called the **domain** of the attribute.
- So a table (relation R) is formally defined as:

  $Table_R = \{<d_1, \ldots d_n> \in D_1 \times \ldots \times D_n \mid R(d_1, \ldots d_n)\}$

  where D_i is the domain of the *i*th-attribute
- Attribute values are required to be **atomic**; that is, indivisible.





Domain Types in SQL

- **char(n).** Fixed length character string, with user-specified length *n*.
- **varchar(n).** Variable length character strings, with user-specified maximum length *n*.
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** Small integer (a machine-dependent subset of the integer domain type).





Domain Types in SQL

- **numeric(p,d).** Fixed-point number, with user-specified precision of *p* digits (sign +,- included), with *d* digits to the right of decimal point.

  Example: **numeric**(3,1), allows 44.5 to be stored exactly, but not 444.5 or 0.32.

- **real, double precision.** Floating-point and double-precision floating-point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.





Create Table Construct (DDL part of SQL)

- An SQL relation is defined using the **create table** command:

create table *r*
(A_1 D_1,
A_2 D_2,
...,
A_n D_n,
(integrity-constraint$_1$),
...,
(integrity-constraint$_k$));

  - *r* is the name of the relation;
  - each A_i is an attribute name in the schema of relation *r*;
  - D_i is the data type of values in the domain of attribute A_i.





Create Table Construct (*cont.*) (DDL part of SQL)

- **Example**:

create table *instructor*
(*ID* **char**(5),
name **varchar**(20),
dept_name **varchar**(20),
salary **numeric**(8,2));





Integrity Constrains in Create Table (DDL part of SQL)

- **not null**
- **primary key** $(A_1, ..., A_n)$
- **foreign key** $(A_1, ..., A_n)$ **references** r

Example:

```
create table instructor
        (ID          char(5),
         name        varchar(20) not null,
         dept_name   varchar(20),
         salary      numeric(8,2),
         primary key (ID),
         foreign key (dept_name) references department;
```

NB: **primary key** declaration on an attribute automatically ensures **not null**



Example (*cont.*)

primary key for "department" in "instructor" (r_1) (referencing relation of the foreign key dependency)

so, "dept_name" is a foreign key from "instructor" referencing "department"

Note: primary key for "instructor"

instructor:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
...	...	...	72000
...	...	Sci.	65000
...	...	...	62000
...	...	Sci.	92000
...	...	...	40000
...	...	...	87000
...	...	...	80000

department:

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

primary key for "department" (r_2) (referenced relation of the foreign key dependency)





Another Example (*variant*)
(DDL part of SQL)

- **create table** *student*
  (*ID* **varchar(5) primary key,**
  name **varchar(20) not null,**
  dept_name **varchar(20),**
  tot_cred **numeric(3,0),**
  foreign key (*dept_name*) **references** *department*);





Alter Table Construct
(DDL part of SQL)

- **Alter Table**
  - Used to add/remove attributes to/from a table.
  - **Example:**

    alter table *r* **add** *A D;*

    where A is the added attribute and D is its domain's type.
  - Variant is: **alter table** *r* **add** *constraint;*

    alter table *r* **drop** *A;*

    where A is the removed attribute.

  - **Note:** All data tuples in the relation *r* are assigned *null* as the value for the new added attribute.





Drop Table Construct
(DDL part of SQL)

- **Drop Table**
  - We use **drop table** to remove a table from the database.
  - **Example:**

```
drop table r;
```

  - **Note:** the command **drop table** *r* is a more drastic action than **delete from** *r*. Why?
  - After a table/relation is dropped, no data tuples can be inserted in it unless it is re-created by the **create table**.



SQL-DML

Data **M**anipulation **L**anguage



Updates to Tables (DML part of SQL)

- **Delete**
  - We use **delete** to delete data tuples from a relation.
  - **Example:**

```
delete from student;
```

  - **Note:** the command **delete from** will delete ALL data tuples from the *student* relation.
  - Other forms of the **delete** command are available (we'll see them later on).





Updates to Tables (DML part of SQL)

- **Insert**
  - We use **insert** to load data tuples into a relation.
  - **Example:**

```
insert into instructor
        values (10211, 'Smith', 'Biology', 66000);
```

  - **Note:** the **values** are specified in the *order* in which the corresponding attributes are listed in the relation schema.



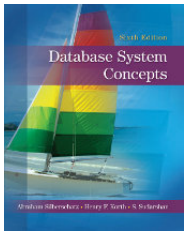
Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)

SQL Queries for Data Retrieval

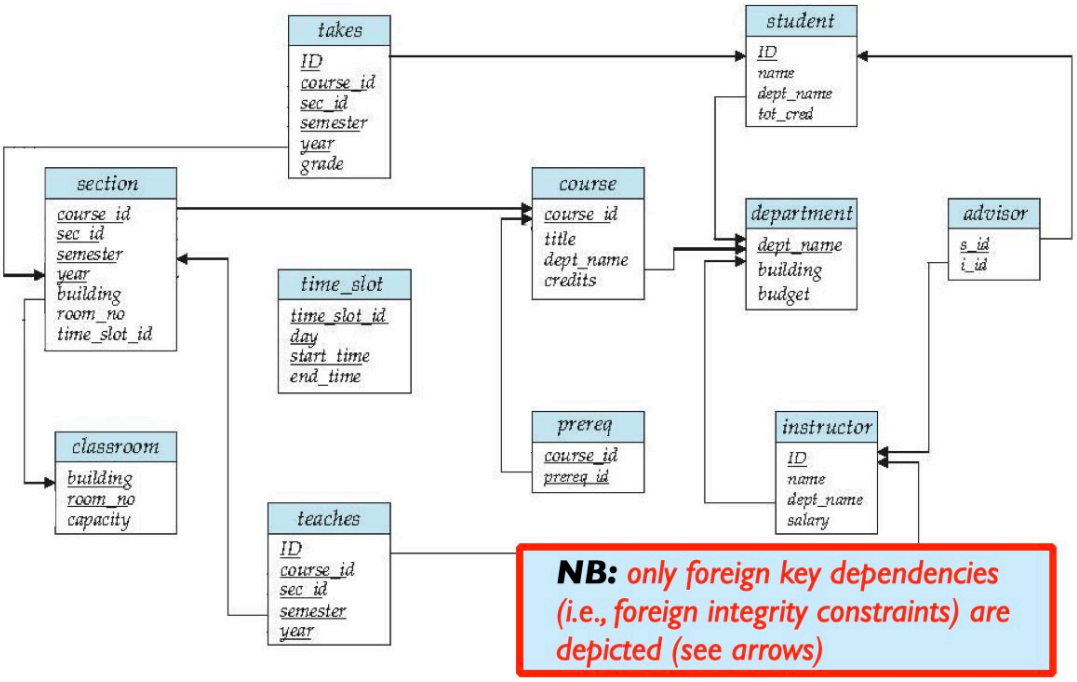
or: How to generate Knowledge from Data and Information

Ch 3, T; section 3.3





Reminder: Schema for University DB







Basic Query Structure
(DML clauses of SQL)

- A typical SQL query has the form:

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P is true;

  - A_i represents an attribute
  - r_i represents a relation
  - P is a predicate over attributes of $r_1, r_2, ..., r_m$.

- Note 1: **select**, **from**, and **where** are called clauses.
- Note 2: the result of a SQL query is a relation!
- Note 3: semicolon " ; " sometimes is missing.



Structure of SQL Queries

Queries on a **Single** Relation



Queries on a Single Relation
The "select" Clause

- The **select** clause lists the attributes desired in the result (i.e., values) of a query
  - corresponds to "Projection" in the (procedural) language of relational algebra.

Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r)$:

A	C
α	1
α	1
β	1
β	2

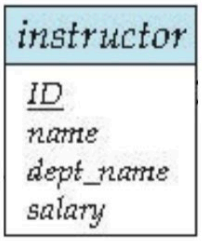
=

A	C
α	1
β	1
β	2



Queries on a Single Relation
The "select" Clause

- **Example**: "Find the names of all instructors":



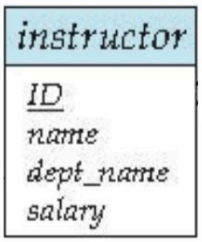




Queries on a Single Relation
The "select" Clause

- **Example**: "Find the names of all instructors":

```
select name
from instructor;
```



- Note: SQL names are case insensitive (i.e., you may use upper- or lower-case letters as you wish.)
  - E.g., *Name* ≡ *NAME* ≡ *name*
  - Some people use upper case wherever here and in the textbook are used bold fonts.





Queries on a Single Relation
The "select" Clause

- **Example:** "Find department names of all instructors":

instructor:

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Result of SQL query:

select *dept_name*
from *instructor;*

"instructor (dept_name)"

Note: *duplicates allowed in SQL queries' result. This contrast to definition of relations as sets*





Queries on a Single Relation
The "select" Clause

- **Example:** "Find department names of all instructors and remove duplicates":





Queries on a Single Relation
The "select" Clause

- **Example**: "Find department names of all instructors and remove duplicates":

```
select distinct dept_name
from instructor;
```

- The optional keyword **all** specifies that duplicates should not be removed (as **select**):

```
select all dept_name
from instructor;
```





Queries on a Single Relation
The "select" Clause

- The **select** clause can contain arithmetic expressions involving the operation, +, –, ∗, and /, and operating on constants or attributes of tuples.
  - **Example**:

```
select ID, name, salary ∗ n
from instructor;
```

  would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is multiplied by n.

  - We can rename "*salary* ∗ *n*" using the **as** clause:

```
select ID, name, salary ∗ n as monthly_salary;
```





Queries on a Single Relation
The "where" Clause

- The **where** clause allows us to select only those rows in the relation of the **from** clause that satisfy a specified predicate P.
  - corresponds to "Selection" in the (procedural) language of relational algebra.

Relation *r* :

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

here the predicate P = P(A,B,D) is:

A=B and D > 5

$\sigma_{A=B \text{ and } D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10





Queries on a Single Relation
The "where" Clause

- **Example**: "Find the names of all instructors in CS department who have salary greater than $70000":





Queries on a Single Relation
The "where" Clause

- **Example**: "Find the names of all instructors in CS department who have salary greater than $70000":

```
select name
from instructor
where dept_name = 'Comp. Sci' and salary > 70000;
```

- Note: the predicate P here is:

  P = P(*dept_name*, *salary*)

  = *dept_name* = 'Comp. Sci' and *salary* > 70000;

  to be read: (*dept_name* = 'Comp. Sci') and (*salary* > 70000)





Queries on a Single Relation
The "where" Clause

- "Find the names of all instructors in CS department who have salary greater than $70000"

Result of SQL query:

```
select name
from instructor
where dept_name = 'Comp. Sci'
      and
      salary > 70000;
```

instructor:

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000





Queries on a Single Relation
The "where" Clause

- Remarks of the predicate P in the clause **where**:

1. P can be formed by using the logical connectives **and, or,** and **not**
2. The operands of the logical connectives can be expressions involving comparison symbols
   = > >= < <=
3. In particular, can be arithmetic expressions.



Structure of SQL Queries

Queries on a **Multiple** Relations



Queries on Multiple Relations Example

- "Find the names of all instructors along with their department names and department building"

department:

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

instructor:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	800(





Queries on Multiple Relations Example

- "Find the names of all instructors along with their department names and department building"

department:

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

instructor:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	800(

SQL query:

select *name, instructor.dept_name, building*
from *instructor, department*
where *instructor.dept_name = department.dept_name;*





Queries on Multiple Relations Example (*cont.*)

- "Find the names of all instructors along with their department names and department building"

Result of SQL query:

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name = department.dept_name;
```

Note on naming convention: "*instructor.dept_name*"

name	*dept_name*	*building*
Srinivasan	Comp. Sci.	Taylor
Wu	Finance	Painter
Mozart	Music	Packard
Einstein	Physics	Watson
El Said	History	Painter
Gold	Physics	Watson
Katz	Comp. Sci.	Taylor
Califieri	History	Painter
Singh	Finance	Painter
Crick	Biology	Watson
Brandt	Comp. Sci.	Taylor
Kim	Elec. Eng.	Taylor



Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)

Cartesian Product on Relations

or: Unconditional Join (“comma”)



Basic Query Structure (DML clauses of SQL)

- A typical SQL query has the form:

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P is true;

  - A_i represents an attribute
  - r_i represents a relation
  - P is a predicate over attributes of $r_1, r_2, ..., r_m$.

- Note 1: **select**, **from**, and **where** are called clauses.
- Note 2: the result of a SQL query is a relation!
- Note 3: semicolon “ ; ” sometimes is missing.





Queries on Multiple Relations The "from" Clause

- The **from** clause lists the relations in the query
  - on two relations, it corresponds to "Cartesian product" in the language of relational algebra.

Relations *r*, *s* :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

Cartesian product of relations r and s

r x *s* :

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b





Queries on Multiple Relations The "from" Clause

- "Find the Cartesian product *instructor* x *teaches*"





Queries on Multiple Relations The "from" Clause

- "Find the Cartesian product *instructor* x *teaches*"

```
select *
from instructor, teaches;
```

  - **NB**: query above generates every possible *instructor–teaches* pairs, with all attributes from both relations.
  - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Remark: Cartesian product not very useful directly, but useful if combined with the **where** clause.





Examples

- "Find the names of all instructors who have taught some course and the course_id"

name	*course_id*
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

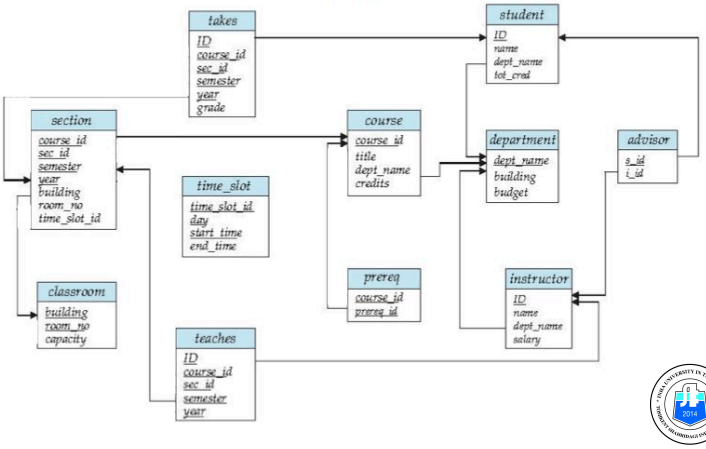




Examples

- "Find the names of all instructors who have taught some course and the course_id"

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101







Examples

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

- "Find the names of all instructors who have taught some course and the course_id"

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*;

instructor
ID
name
dept_name
salary

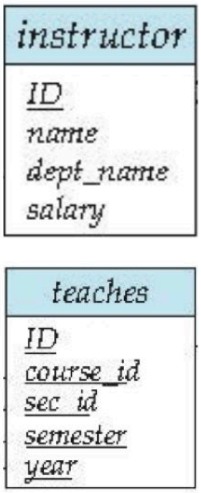
teaches
ID
course_id
sec_id
semester
year

- "Find the names of all instructors in the Art department who have taught some course and the course_id"





Examples



- "Find the names of all instructors who have taught some course and the course_id"

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

- "Find the names of all instructors in the Art department who have taught some course and the course_id"

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
      and instructor. dept_name = 'Art';
```

Note on *instructor.* : not necessary but useful ("good" coding' style)





Queries on Multiple Relations The "from" Clause

- In general, the meaning of a SQL query from two relations *r* and *s* can be understood as follows:

1. Generate the Cartesian product of the relations *r* and *s* listed in the **from** clause (i.e., compute *r* x *s*).
2. On the attributes of the result of step 1, apply the predicate P specified in the **where** clause.
3. For each tuple in the result of step 2, output the value of attributes specified in the **select** clause.



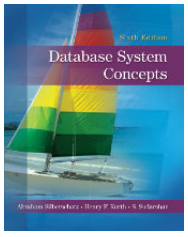
Outline

1. From Previous Week
2. Query Languages
3. SQL: The Basic Elements
4. Structure of SQL Queries for Data Retrieval
5. Cartesian Product
6. The Natural Join
7. Modification of the Database
8. In-Class Quiz #1 (Lecture 2)

The Natural Join

or: How *not* to use the Cartesian Product!

Ch 3, T; subsection 3.3.3





Reminder: Relational Operations

- We concentrate on the following operations:
  - selection of tuples/rows,
  - projection (or: selection of attributes/columns),
  - natural join,
  - Cartesian product,
  - union,
  - set difference,
  - intersection.

- In blue are "derived" operations, that is, operations defined on other "primary" operations (in red).





Natural Join: Example

- Relations *r*, *s*:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

- *r* + *s*:

ID	name	salary	dept_name	building	budget
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
12121	Wu	90000	Finance	Painter	120000
15151	Mozart	40000	Music	Packard	80000
22222	Einstein	95000	Physics	Watson	70000
32343	El Said	60000	History	Painter	50000
33456	Gold	87000	Physics	Watson	70000
45565	Katz	75000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
76543	Singh	80000	Finance	Painter	120000
76766	Crick	72000	Biology	Watson	90000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000





Another Example: *instructor* and *teaches*

- Relations *r*, *s*:

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches

The only common attribute

- *r* + *s*:

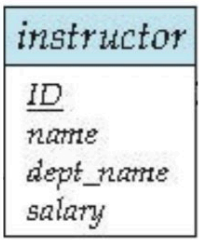
ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

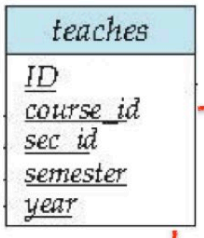




The "natural join" Clause

- "For all instructors who have taught some course [note: so you don't want all instructors!], find their names and course ID of all courses they taught"





- Answer by using Cartesian product "controlled" by proper **where** is:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

- Answer by Natural Join is:

```
select name, course_id
from instructor natural join teaches;
```





More on Natural Join

- A SQL query can be of this form:

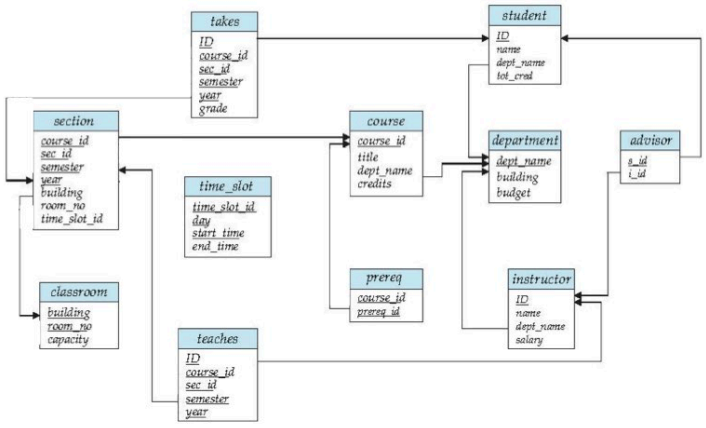
select $A_1, A_2, ..., A_n$
from r_1 **natural join** r_2 ... **natural join** r_m
where P on attributes in r_1 r_2 ... r_m is true;





More on Natural Join

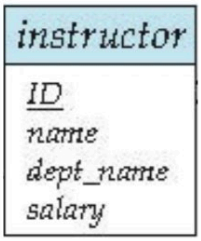
- **Example**: "List the names of instructors along with the title of courses that they teach".

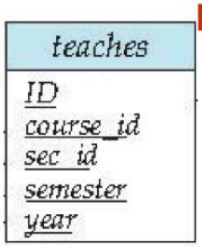


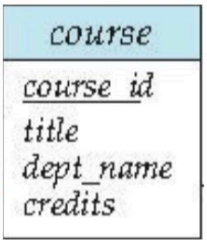




More on Natural Join







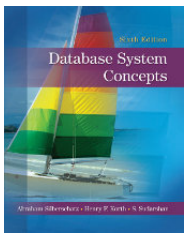
- **Example**: "List the names of instructors along with the title of courses that they teach"

select *name, title*
from *instructor* **natural join** *teaches, course*
where *teaches.course_id = course.course_id*;



Join Conditions

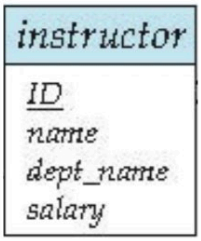
Ch 4, T; section 4.1

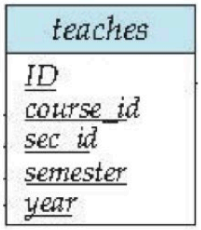




Example (*cont.*) - The "using" Clause

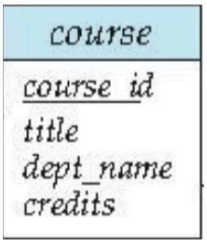
- "List names of instructors along with title of courses that they teach"





- Answer by using Natural Join plus Cartesian product "controlled" by proper **where** is:

select *name, title*
from *instructor* **natural join** *teaches, course*
where *teaches.course_id = course.course_id*;



- Answer by a sequence of joins with new clause **using** is:

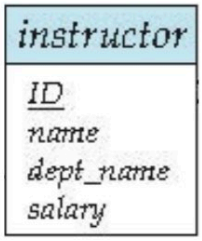
select *name, title*
from *instructor* **natural join** *teaches* **join** *course* **using** *course_id*;

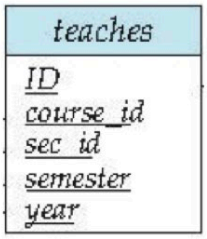


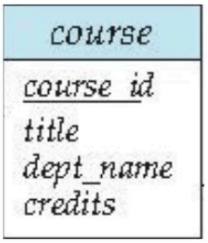


Important Remarks

- "List names of instructors along with title of courses that they teach"







select *name, title*
from *instructor* **natural join** *teaches, course*
where *teaches.course_id = course.course_id*;

select *name, title*
from *instructor* **natural join** *teaches* **join** *course* **using** *course_id*;

select *name, title*
from *instructor* **natural join** *teaches* **natural join** *course*;

NB: *conceptually different*

This is a wrong SQL code for the English query above!





Important Remarks

- "List all data of instructors along with title of courses that they teach"

NB1: *not equivalent in schema!*

```
select *
from instructor natural join teaches, course
where teaches.course_id = course.course_id;

select *
from instructor natural join teaches join course using course_id;

select *
from instructor natural join teaches natural join course;
```

NB2: *conceptually different*

This is a wrong SQL code for the English query above!

instructor(ID, name, dept_name, salary)

course(course_id, title, dept_name, credits)

teaches(ID, course_id, sec_id, semester, year)



Join Conditions - "on"

student(ID, name, dept_name, tot_cred)

- Consider the following SQL query:

```
select *
from student join takes on student.ID = takes.ID;
```

- It's similar to the following SQL query:

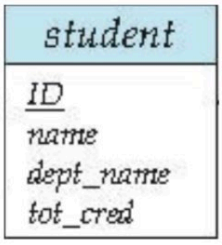
```
select *
from student natural join takes;
```

with the only difference of duplicating the *ID*'s column attributes in the resulting relation.

takes(ID, course_id, sec_id, semester, year, grade)



Join Conditions - "on"



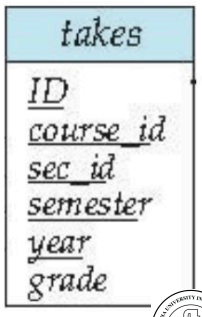
- Reconsider the SQL query:

```
select *
from student join takes on student.ID = takes.ID;
```

- It's equivalent to the following SQL query:

```
select *
from student, takes
where student.ID = takes.ID;
```

- We can avoid duplication of *IDs* by the clause **as**.





join…on + as

```
select * as ID, name, dept_name, tot_cred, course_id, sec_id, semester, year, grade
from student join takes on student.ID = takes.ID;
```

ID	name	dept_name	tot_cred	course_id	sec_id	semester	year	grade
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2010	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2009	A
19991	Brandt	History	80	HIS-351	1	Spring	2010	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	*null*





Join Expressions

- **Join operations** (in form of join expressions) take two relations and return as a result another relation.
- **Def:** A **join** operation is a Cartesian product A x B which requires that tuples in the relations A and B match *under some condition*.
  - A join operation always specify the attributes that are present in the result of the join.
- The join operations are typically used as subquery expressions in the **from** clause.



We'll see more on Join Expressions.

Outline

1 From Previous Week

2 Query Languages

3 SQL: The Basic Elements

4 Structure of SQL Queries for Data Retrieval

5 Cartesian Product

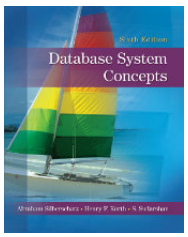
6 The Natural Join

7 Modification of the Database

8 In-Class Quiz #1 (Lecture 2)

Modification of the Database

Ch 3, T; section 3.9





Deletion

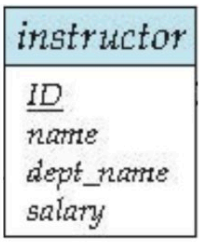


- "Delete all instructors"

  delete from *instructor*

- "Delete all instructors from the Finance department"

  delete from *instructor*
  where *dept_name*= 'Finance';





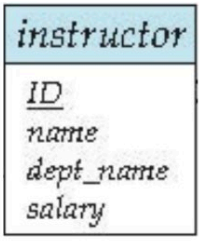


Deletion: Example



- "Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building"

  ?





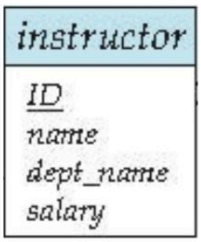


Example (*cont.*)



- "Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building"

```
delete from instructor
where dept_name in (select dept_name
                    from department
                    where building = 'Watson');
```







Deletion : An Issue (*to be careful about*)

- "Delete all instructors whose salary is less than the average salary of instructors"

```
delete from instructor
where salary < (select avg (salary)
                from instructor);
```

I.e., check **where** condition

- Problem: as we delete tuples from *instructor*, the average salary changes
- Solution used in SQL:
  1. First, compute **avg** (salary) and find all tuples to delete
  2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)





Insertion (of plain data)

- "Add a new tuple to *course*"

```
insert into course
        values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- or equivalently

course's attributes repeated for the shake of remembering

```
insert into course (course_id, title, dept_name, credits)
        values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- "Add a new tuple to *student* with *tot_creds* set to *null*"

```
insert into student (ID, name, dept_name, tot_credits)
        values ('3003', 'Green', 'Finance', null);
```





Insertion (of plain data)

- To insert partial data… see SQL manual…
- For instance, *mySQL*:

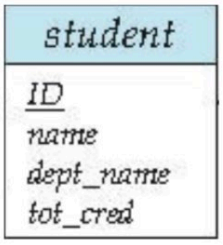
```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
    [INTO] tbl_name
    [(col_name [, col_name] ...)]
    {VALUES | VALUE} (value_list) [, (value_list)] ...
```

Source: https://dev.mysql.com/doc/refman/5.5/en/insert.html





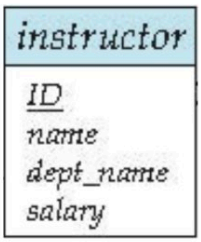
Insertion (from other table)



- What is the meaning of following query?

```
insert into instructor
        select ID, name, dept_name, 20000
        from student
        where dept_name = 'Comp.Sci' and tot_cred > 145;
```

- **Note 1**: "**select**" allows to insert a set of tuples.
- **Note 2**: "**select**" must be evaluated before insertions.
  (otherwise an *insertion loop* arises)




```
insert into student
        select *
        from student;
```





Updates (to change *some* values)

- "Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by a 5%"

  ?





Updates (to change *some* values)

- "Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by a 5%"
  - We can write two **update** statements in sequence:

```
update instructor
set salary = salary * 1.03
where salary > 100000;

update instructor
set salary = salary * 1.05
where salary <= 100000;
```

  - The order of above updates is important! (Why?)
  - Can be done better using the **case** statement.





"Case" Statement for *Conditional* Updates

- "Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by a 5%"

```
update instructor
set salary = case
                when salary <= 100000 then salary * 1.05
                else salary * 1.03
             end;
```





Updates with Scalar Subqueries

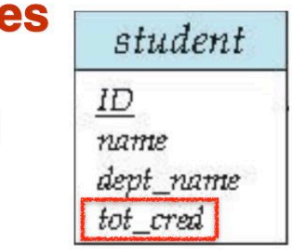
- Recompute and update *tot_creds* value for all students:

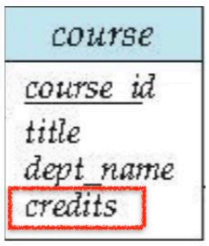
S: correlation name

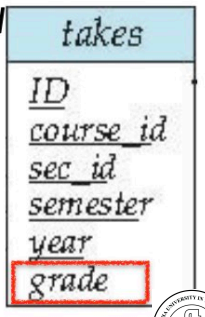
ID: correlation variable

```
update student S
set tot_cred = (select sum(credits)
                from takes, course
                where takes.course_id = course.course_id
                    and S.ID = takes.ID
                    and takes.grade <> 'F'
                    and takes.grade is not null);
```

- Update sets *tot_creds* to *null* if *takes.grade* is *null*.









Outline

1 From Previous Week
2 Query Languages
3 SQL: The Basic Elements
4 Structure of SQL Queries for Data Retrieval
5 Cartesian Product
6 The Natural Join
7 Modification of the Database
8 In-Class Quiz #1 (Lecture 2)

In-Class Quiz #1

(LECTURE 2)

Only for students who submitted Homework Assignment #1.

No slides provided for Quiz #1



Summary

1 From Previous Week

2 Query Languages

3 SQL: The Basic Elements

4 Structure of SQL Queries for Data Retrieval

5 Cartesian Product

6 The Natural Join

7 Modification of the Database

8 In-Class Quiz #1 (Lecture 2)



DATABASE

End of Slides

WEEK 5

Code: W5.L1L2[QUIZ#1]